

Week-3:

Build a **feed forward neural network** for prediction of **logic gates**.

❖ **Feed-Forward Neural Network**

- ❖ A Feed-Forward Neural Network is the foundational architecture in deep learning that allows **machines to learn from data by passing information forward through layers of interconnected neurons**.
- ❖ It is versatile and widely used in various domains for its simplicity and effectiveness in capturing complex patterns in data.
- ❖ A Feed-Forward Neural Network (FFNN) is a type of artificial neural network where connections between the nodes do not form any cycles.
- ❖ It is the simplest and most common type of neural network used for a variety of machine learning tasks, such as classification, regression, and function approximation.
- ❖ **In a feed-forward neural network, data flows in one direction: from the input layer, through one or more hidden layers, to the output layer.** There are no loops or cycles in this flow of information. **The network is called "feed-forward" because each layer feeds its output to the next layer, and there is no feedback to earlier layers.**

Basic Structure of a Feed-Forward Neural Network:

- ❖ **Input Layer:** Receives the input data. Each neuron in this layer represents a feature of the input data
- ❖ **Hidden Layers:** These are intermediate layers between the input and output layers. They are called "hidden" because they do not directly interact with the outside environment. The neurons in these layers apply weights and biases to the input data and use activation functions to transform the data into a form that the network can learn from.
- ❖ **Output Layer:** Produces the final output of the network. For a classification task, the output layer might contain neurons representing different classes, and the values of these neurons represent the probabilities of each class.

- ❖ In forward propagation, the **input data passes through the network layer by layer**. Each layer transforms the input data using weights, biases, and activation functions. This process continues until the data reaches the output layer. The output of the network is then compared with the actual target values (labels) to calculate the loss or error.
- ❖ Back-propagation is the **process of adjusting the weights and biases in the network to minimize the error (loss)**. It involves calculating the gradient of the loss function with respect to each weight by applying the chain rule of calculus. The gradients are then used to update the weights and biases using optimization algorithms like Stochastic Gradient Descent (SGD) or Adam.

Advantages:

- ❖ Simple and easy to understand.
- ❖ Effective for a wide range of problems, especially when data is well-structured.
- ❖ Suitable for both classification and regression tasks.

Aim of the Program:

- ❖ To design and implement feed-forward neural networks using TensorFlow/Keras to accurately learn and predict the behavior of basic logic gates—AND, OR, XOR, and NOT—through supervised learning and model training.

TensorFlow installed in your Python environment

1. pip install tensorflow

Import Libraries: Import the necessary libraries.

1. import numpy as np

2. from tensorflow.keras.models import Sequential

3. from tensorflow.keras.layers import Dense

4. from sklearn.metrics import accuracy_score, f1_score

Step 1: Prepare the Data

Input data for 2-input logic gates (AND, OR, XOR)

1. X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

Outputs for each logic gate

- 1. `Y_AND = np.array([[0], [0], [0], [1]])` # AND Gate**
- 2. `Y_OR = np.array([[0], [1], [1], [1]])` # OR Gate**
- 3. `Y_XOR = np.array([[0], [1], [1], [0]])` # XOR Gate**

NOT Gate input (1-input data)

- 1. `X_NOT = np.array([[0], [1]])`**
- 2. `Y_NOT = np.array([[1], [0]])`**

Step 2: Define the Feed-Forward Neural Network Model

1. **def build_ffnn(input_dim):**

2. **model = Sequential()**

Input Layer and first hidden layer

1. **model.add(Dense(4, input_dim=input_dim, activation='relu'))**

Second hidden layer

1. **model.add(Dense(4, activation='relu'))**

Output layer

1. **model.add(Dense(1, activation='sigmoid'))**

Compile the model

1. **model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])**

2. **return model**

Build models for each gate

- 1. model_and = build_ffnn(2)**
- 2. model_or = build_ffnn(2)**
- 3. model_xor = build_ffnn(2)**

Train each model

- 1. model_and.fit(X, Y_AND, epochs=100, verbose=0)**
- 2. model_or.fit(X, Y_OR, epochs=100, verbose=0)**
- 3. model_xor.fit(X, Y_XOR, epochs=100, verbose=0)**

NOT gate model needs 1 input

- 1. model_not = build_ffnn(1)**
- 2. model_not.fit(X_NOT, Y_NOT, epochs=100, verbose=0)**

- ❖ The verbose parameter in the **.fit method of a Keras model** controls the amount of information displayed in the console while the model is being trained.
- ❖ It determines how much feedback you get during the training process. This feedback can help you monitor the training progress, understand how the model is performing over time, and diagnose potential issues.

verbose Options

- ❖ There are three primary options for the verbose parameter:
- ❖ **verbose=0: Silent Mode**
- ❖ No output is printed to the console during training.
- ❖ This mode is useful when you want to suppress all training output, such as when running a script in the background, logging output to a file, or if you do not need to monitor the training progress.

❖ **verbose=1: Progress Bar Mode (Default)**

❖ Displays a dynamic progress bar in the console, showing the progress of each epoch (one complete pass through the entire training dataset).

❖ **The progress bar includes:**

- ❖ The current epoch number and total number of epochs.
- ❖ The number of samples processed and the total number of samples.
- ❖ The elapsed time for the epoch.
- ❖ Metrics like loss and accuracy at the end of the epoch.
- ❖ This is the default setting and provides a good balance of information and readability.

Epoch 1/100

4/4 [=====] - 0s 998us/step - loss: 0.6932 - accuracy: 0.5000

Epoch 2/100

4/4 [=====] - 0s 1ms/step - loss: 0.6927 - accuracy: 0.7500

verbose=2: One Line per Epoch Mode

- ❖ Prints a single line for each epoch with information about the epoch number, loss, and metrics.
- ❖ Does not show a progress bar, but instead shows only the summary information.
- ❖ This mode is useful if you want to log training information in a simpler, more readable format or if your console does not support dynamic progress bars.

Example output for verbose=2:

Epoch 1/100 - loss: 0.6932 - accuracy: 0.5000

Epoch 2/100 - loss: 0.6927 - accuracy: 0.7500

When to Use Each verbose Setting

❖ **verbose=0:**

- ❖ Use when you don't need to see any output during training or when you are running many models and want to suppress output for simplicity.
- ❖ Ideal for automated runs or background processes where output is not required for monitoring.

❖ **verbose=1:**

- ❖ Best for interactive sessions where you want to see a progress bar and get immediate feedback on training.
- ❖ Useful for notebooks or terminals that support dynamic updates.

❖ **verbose=2:**

- ❖ Ideal if you prefer a clean, textual log output that is less visually intensive than a progress bar.
- ❖ Good for logging output to files where each epoch's result needs to be recorded clearly.

Step 3: Evaluate and Predict Part 1

1. **# AND Gate Evaluation**
2. **pred_and = np.round(model_and.predict(X)).astype(int)**
3. **print("AND Gate Predictions:")**
4. **print(pred_and)**
5. **print("Accuracy:", accuracy_score(Y_AND, pred_and))**
6. **print("F1 Score:", f1_score(Y_AND, pred_and))**
7. **# OR Gate Evaluation**
8. **pred_or = np.round(model_or.predict(X)).astype(int)**
9. **print("\nOR Gate Predictions:")**
10. **print(pred_or)**
11. **print("Accuracy:", accuracy_score(Y_OR, pred_or))**
12. **print("F1 Score:", f1_score(Y_OR, pred_or))**

Step 3: Evaluate and Predict Part 2

1. # XOR Gate Evaluation
2. `pred_xor = np.round(model_xor.predict(X)).astype(int)`
3. `print("\nXOR Gate Predictions:")`
4. `print(pred_xor)`
5. `print("Accuracy:", accuracy_score(Y_XOR, pred_xor))`
6. `print("F1 Score:", f1_score(Y_XOR, pred_xor))`

7. # NOT Gate Evaluation
8. `pred_not = np.round(model_not.predict(X_NOT)).astype(int)`
9. `print("\nNOT Gate Predictions:")`
10. `print(pred_not)`
11. `print("Accuracy:", accuracy_score(Y_NOT, pred_not))`
12. `print("F1 Score:", f1_score(Y_NOT, pred_not))`

1/1 [=====] - 0s 47ms/step

AND Gate Predictions:

[[0]
[0]
[0]
[1]]

Accuracy: 1.0

F1 Score: 1.0

1/1 [=====] - 0s 42ms/step

OR Gate Predictions:

[[1]
[1]
[1]
[1]]

Accuracy: 0.75

F1 Score: 0.8571428571428571

1/1 [=====] - 0s 53ms/step

XOR Gate Predictions:

[[1]

[1]

[0]

[0]]

Accuracy: 0.5

F1 Score: 0.5

1/1 [=====] - 0s 45ms/step

NOT Gate Predictions:

[[1]

[0]]

Accuracy: 1.0

F1 Score: 1.0

For inputs:

AND GATE

- $[0, 0] \rightarrow 0$
- $[0, 1] \rightarrow 0$
- $[1, 0] \rightarrow 0$
- $[1, 1] \rightarrow 1$ (Only 1 AND 1 gives 1)

For inputs:

OR GATE

- $[0, 0] \rightarrow 0$
- $[0, 1] \rightarrow 1$
- $[1, 0] \rightarrow 1$
- $[1, 1] \rightarrow 1$ (Any 1 gives 1)

For inputs:

XOR GATE

- $[0, 0] \rightarrow 0$
- $[0, 1] \rightarrow 1$
- $[1, 0] \rightarrow 1$
- $[1, 1] \rightarrow 0$ (Output is 1 only when inputs are different)

NOT GATE

For inputs:

- $[0] \rightarrow 1$
- $[1] \rightarrow 0$ (NOT inverts the input)

Summary

- ❖ Conclusion: This program illustrates the process of building and training feed-forward neural networks using TensorFlow/Keras to predict the outputs of basic logic gates—AND, OR, XOR, and NOT. It involves preparing input-output datasets for each gate, defining a neural network model with two hidden layers using ReLU activation and an output layer with a sigmoid function. Each model is trained for 100 epochs with its corresponding dataset. After training, the program evaluates the models by generating and displaying predictions based on the input values for each logic gate.